🏗️ Heavy Industrial Machinery — Price Prediction

MLP Kaggle Competition | Complete ML Pipeline

Problem: Predict `TargetValue` (auction sale price in USD) of heavy machinery
Metric: RMSLE ≤ **0.15** (Root Mean Squared Logarithmic Error)
Data: 138 701 train rows | 15 000 test rows | 50 features

`Load` → `EDA` → `Preprocessing` → `Feature Eng.` → `Models` → `HPT` → `Error Analysis` → `Submit`

In [2]:
```
# This Python 3 environment comes with many helpful analytics libraries instal
# It is defined by the kaggle/python Docker image: https://github.com/kaggle/d
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will lis

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/) that ge
# You can also write temporary files to /kaggle/temp/, but they won't be saved

# Use the kagglehub client library to attach Kaggle resources like competition
# Learn more about kagglehub: https://github.com/Kaggle/kagglehub/blob/main/RE

import kagglehub
# kagglehub.dataset_download('<owner>/<dataset-slug>')
```

```
/kaggle/input/competitions/heavy-equipment-selling-price-prediction-challenge/s
ample_submission.csv
/kaggle/input/competitions/heavy-equipment-selling-price-prediction-challenge/t
rain.csv
/kaggle/input/competitions/heavy-equipment-selling-price-prediction-challenge/m
etadata.csv
/kaggle/input/competitions/heavy-equipment-selling-price-prediction-challenge/t
est.csv
```

In [3]:
```
# ═══════════════════════════════════════════════════
# 1.  SETUP & IMPORTS
# ═══════════════════════════════════════════════════
import numpy as np
```

```
import pandas as pd
import warnings, os
warnings.filterwarnings('ignore')
pd.set_option('display.max_columns', 60)
pd.set_option('display.float_format', '{:.2f}'.format)

import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import seaborn as sns
import missingno as msno

plt.rcParams['figure.figsize'] = (14, 5)
plt.rcParams['font.size'] = 11
sns.set_theme(style='whitegrid', palette='Set2')

from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OrdinalEncoder
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split, KFold, RandomizedSearchC
from sklearn.metrics import mean_squared_error, r2_score, make_scorer
from sklearn.ensemble import RandomForestRegressor, HistGradientBoostingRegres
from sklearn.linear_model import Ridge
from sklearn.dummy import DummyRegressor
from xgboost import XGBRegressor
import lightgbm as lgb

SEED = 42
np.random.seed(SEED)
print("✅ All libraries loaded!")
print(f"   NumPy {np.__version__} | Pandas {pd.__version__}")
```

```
✅ All libraries loaded!
   NumPy 2.4.6 | Pandas 2.3.3
```

In [4]:

```
# ═══════════════════════════════════════════════════════
# 2.  DATA LOADING
# ═══════════════════════════════════════════════════════
train_df = pd.read_csv('/kaggle/input/competitions/heavy-equipment-selling-pri
test_df  = pd.read_csv('/kaggle/input/competitions/heavy-equipment-selling-pri
sample   = pd.read_csv('/kaggle/input/competitions/heavy-equipment-selling-pri
metadata = pd.read_csv('/kaggle/input/competitions/heavy-equipment-selling-pri

print("=" * 60)
print(f"  TRAIN   : {train_df.shape[0]:,} rows × {train_df.shape[1]} cols")
print(f"  TEST    : {test_df.shape[0]:,}  rows × {test_df.shape[1]} cols")
print(f"  SAMPLE  : {sample.shape}")
print("=" * 60)

print("\nRequired submission format:")
display(sample.head(3))
print("\nColumn descriptions (metadata):")
print(metadata[['Variable','Rephrased Description']].to_string(index=False))
```

```
============================================================
  TRAIN   : 138,701 rows × 50 cols
  TEST    : 15,000  rows × 49 cols
  SAMPLE  : (15000, 2)
============================================================
```

Required submission format:

	TransactionID	TargetValue
0	1139307	44674.90
1	1139419	91450.10
2	1139482	76618.10

Column descriptions (metadata):

Variable	Rephrased Description
TransactionID	A unique serial number assigned to each individual accounting record.
TargetValue	The financial amount of the transaction, denominated in US Dollars.
AssetID	The internal tracking number assigned to a specific piece of hardware.
ProductConfigID	A system-generated code representing the technical build-out of the unit.
DataOriginCode	A code indicating the specific software or platform where the data originated.
VendorPartnerID	The unique ID of the external entity or third-party provider involved.
TransactionDate	The official calendar date when the entry was finalized in the system.
RegionCode	The specific geographic area or legal jurisdiction where the event occurred.
ManufactureYear	The year the asset was originally produced.
OperationalHoursMeter	The lifetime total of active run-time or work cycles logged by the machine.
UtilizationTier	A ranking (Low, Medium, or High) based on how heavily the asset is used.
Spec_FullDescriptor	The complete alphanumeric string detailing all technical parameters.
Spec_BaseClass	The broad, primary category for the asset's technical specifications.
Spec_SubClass	A secondary, more specific tier within the technical classification.
Spec_ReleaseSeries	Information regarding the specific production run or version number.
Spec_VariantModifier	A code identifying specific revisions or custom tweaks to a standard model.
FunctionalClassification	A secondary grouping based on the asset's operational role.
AssetScaleFactor	A value representing the physical dimensions or capacity relative to a standard unit.
InventoryGroupCategory	The highest level of classification within the inventory system.
InventoryGroupDescription	A detailed text explanation of the primary inventory category.
CabinType	The specific design of the operator’s station and environmental enclosure.
DrivetrainType	The mechanical system used to transfer power to the movement components.
Forks	The specific attachment module used for lifting (Legacy terminology).
col1	The primary configuration of the drive system.
col3	The type of subsystem used for machine stabilization.
col4	The specific layou

t or reach of the extension arm.	
col5	The status or type of the engin
e’s air intake/aspiration system.	
col6	The current status or type
of the primary extension module.	
col7	The horizontal width
of the active working component.	
col8	Specifications regarding the machine's
protective housing or shielding.	
col9	The total rated power output
(e.g., horsepower or kilowatts).	
col10	The classification of the hydrau
lic or fluid-based power system.	
col11	The type of interface used for lo
ad-bearing and shock absorption.	
col12	Details on accessories designed for pi
ercing or penetrating materials.	
col13	Details on accessories used for
finishing or smoothing surfaces.	
col14	The logic and hardware us
ed to control machine actuators.	
col15	The external meas
urements of the wheels or tires.	
col16	The type of mechanism used to coup
le or join different interfaces.	
col18	The system used for tract
ion and contact with the ground.	
col19	The rate of fluid fl
ow through the system by volume.	
col20	The specific tread or surface
pattern of the traction system.	
col21	The width of the surface area tha
t makes contact with the ground.	
col22	The maximum reach o
r length of the operational arm.	
col23	The design of the interface u
sed to hold or retain materials.	
col24	The configuration map for the
machine’s control system logic.	
col25	The specific profile or s
hape of the traction components.	
col27	The specific design or cat
egory of the attached work tool.	
col28	The configuration of the operato
r's directional steering inputs.	
col29	The type of different
ial used for power distribution.	
col30	The primary interface us
ed by the operator for steering.	

2. Data Loading & Overview

In [5]:
```
print("First 5 rows of training data:")
display(train_df.head(3))
print()
train_df.info(verbose=True, show_counts=True)
```

```
First 5 rows of training data:
```

	TransactionID	TargetValue	AssetID	ProductConfigID	DataOriginCode	Vendor
0	1139309	57000.00	117665	88	ACH138	
1	1139316	26500.00	1001282	4616	ACH138	
2	1139317	21000.00	772709	1948	ACH138	

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 138701 entries, 0 to 138700
Data columns (total 50 columns):
 #   Column                      Non-Null Count   Dtype  
---  ------                      --------------   -----  
 0   TransactionID               138701 non-null  int64  
 1   TargetValue                 138701 non-null  float64
 2   AssetID                     138701 non-null  int64  
 3   ProductConfigID             138701 non-null  int64  
 4   DataOriginCode              138701 non-null  object 
 5   VendorPartnerID             138701 non-null  object 
 6   ManufactureYear             138701 non-null  int64  
 7   OperationalHoursMeter       82058 non-null   float64
 8   UtilizationTier             50475 non-null   object 
 9   TransactionDate             138701 non-null  object 
 10  Spec_FullDescriptor         138701 non-null  object 
 11  Spec_BaseClass              138701 non-null  object 
 12  Spec_SubClass               101716 non-null  object 
 13  Spec_ReleaseSeries          32896 non-null   object 
 14  Spec_VariantModifier        52332 non-null   object 
 15  AssetScaleFactor            97491 non-null   object 
 16  FunctionalClassification    138701 non-null  object 
 17  RegionCode                  138701 non-null  object 
 18  InventoryGroupCategory      138701 non-null  object 
 19  InventoryGroupDescription   138701 non-null  object 
 20  col1                        33786 non-null   object 
 21  CabinType                   138701 non-null  object 
 22  Forks                       29319 non-null   object 
 23  col3                        29253 non-null   object 
 24  col4                        10381 non-null   object 
 25  DrivetrainType              48745 non-null   object 
 26  col5                        10381 non-null   object 
 27  col6                        23405 non-null   object 
 28  col7                        23405 non-null   object 
 29  col8                        23405 non-null   object 
 30  col9                        23405 non-null   object 
 31  col10                       128320 non-null  object 
 32  col11                       23405 non-null   object 
 33  col12                       38364 non-null   object 
 34  col13                       23405 non-null   object 
 35  col14                       23405 non-null   object 
 36  col15                       42277 non-null   object 
 37  col16                       89956 non-null   object 
 38  col18                       66 non-null      object 
 39  col19                       66 non-null      object 
 40  col20                       71016 non-null   object 
 41  col21                       71016 non-null   object 
 42  col22                       71018 non-null   object 
 43  col23                       71018 non-null   object 
 44  col24                       71018 non-null   object 
 45  col25                       71016 non-null   object 
 46  col27                       14959 non-null   object 
 47  col28                       14959 non-null   object 
 48  col29                       18872 non-null   object 
```

```
 49  col30                      18872 non-null   object
dtypes: float64(2), int64(4), object(44)
memory usage: 52.9+ MB
```

In [6]:
```
print("\nTrain Datatset shape:")
display(train_df.shape)

print("\nTest Datatset shape:")
test_df.shape
```

```
Train Datatset shape:
(138701, 50)
Test Datatset shape:
```

Out[6]: (15000, 49)

In [7]:
```
print("Numerical summary statistics:")
display(train_df.describe())
```

```
Numerical summary statistics:
```

	TransactionID	TargetValue	AssetID	ProductConfigID	ManufactureYear
count	138701.00	138701.00	138701.00	138701.00	138701.00
mean	2186275.95	41521.87	1204814.32	7286.42	1891.64
std	1128496.62	26361.13	528737.59	7542.42	306.99
min	1139309.00	7500.00	8.00	39.00	1001.00
25%	1465754.00	20500.00	1007930.00	1693.00	1990.00
50%	1751884.00	35000.00	1246956.00	3852.00	1998.00
75%	2372139.00	57000.00	1495817.00	11873.00	2004.00
max	6333410.00	142000.00	2478476.00	37209.00	2015.00

In [8]:
```
print("How Many Coloum Present in Train Dataset")
train_df.columns
```

```
How Many Coloum Present in Train Dataset
```

Out[8]:
```
Index(['TransactionID', 'TargetValue', 'AssetID', 'ProductConfigID',
       'DataOriginCode', 'VendorPartnerID', 'ManufactureYear',
       'OperationalHoursMeter', 'UtilizationTier', 'TransactionDate',
       'Spec_FullDescriptor', 'Spec_BaseClass', 'Spec_SubClass',
       'Spec_ReleaseSeries', 'Spec_VariantModifier', 'AssetScaleFactor',
       'FunctionalClassification', 'RegionCode', 'InventoryGroupCategory',
       'InventoryGroupDescription', 'col1', 'CabinType', 'Forks', 'col3',
       'col4', 'DrivetrainType', 'col5', 'col6', 'col7', 'col8', 'col9',
       'col10', 'col11', 'col12', 'col13', 'col14', 'col15', 'col16', 'col1
8',
       'col19', 'col20', 'col21', 'col22', 'col23', 'col24', 'col25', 'col2
7',
       'col28', 'col29', 'col30'],
      dtype='object')
```

In [9]:
```
print("Know about Submission Columns")
sample.columns  # We name Submission File to sample
```

```
Know about Submission Columns
```

Out[9]:
```
Index(['TransactionID', 'TargetValue'], dtype='object')
```

In [10]:
```
# # 5. Create a Pandas DataFrame for the submission file
# # submission = pd.DataFrame({
# submission = pd.DataFrame({
#     'TransactionID': test_df['TransactionID'],   # ← Key fix: use test IDs,
#     'TargetValue': train_df['TargetValue'].mean()       # Dummy baseline = m
# })
# submission.to_csv('submission.csv', index=False)
# print("Done!")
# print(submission.head())
```

```
Done!
   TransactionID  TargetValue
0        1139307     41521.87
1        1139419     41521.87
2        1139482     41521.87
3        1139522     41521.87
4        1139684     41521.87
```

3. Exploratory Data Analysis (EDA)

Key objectives:

1. **Target distribution** — understand price range & skewness
2. **Missing values** — scale and pattern of absent data
3. **Categorical features** — category counts and price impact
4. **Numerical features** — distributions and target correlation
5. **Bivariate analysis** — how features relate to price

> **RMSLE insight:** We train on `log1p(TargetValue)` so proportional errors at all price levels are penalised equally.

In [11]:
```
# ═══════════════════════════════════════════════════
# 3.1  TARGET VARIABLE
# ═══════════════════════════════════════════════════
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
fig.suptitle('Target Variable: TargetValue (Sale Price USD)', fontsize=13, for

log_target = np.log1p(train_df['TargetValue'])

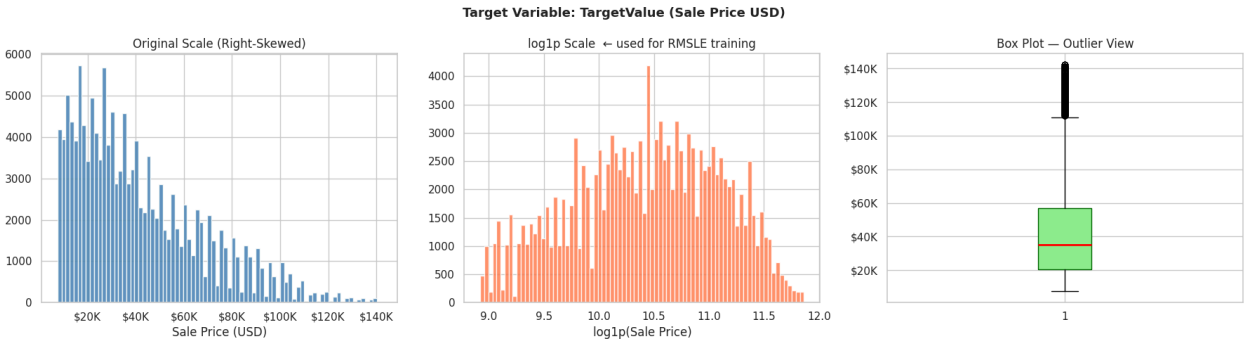
axes[0].hist(train_df['TargetValue'], bins=80, color='steelblue', edgecolor='w
axes[0].set_title('Original Scale (Right-Skewed)')
axes[0].set_xlabel('Sale Price (USD)')
axes[0].xaxis.set_major_formatter(mticker.FuncFormatter(lambda x,_: f'${x/1000
```

```
axes[1].hist(log_target, bins=80, color='coral', edgecolor='white', alpha=.85)
axes[1].set_title('log1p Scale  ← used for RMSLE training')
axes[1].set_xlabel('log1p(Sale Price)')

bp = axes[2].boxplot(train_df['TargetValue'], patch_artist=True,
                     boxprops=dict(facecolor='#90EE90', color='darkgreen'),
                     medianprops=dict(color='red', linewidth=2))
axes[2].set_title('Box Plot — Outlier View')
axes[2].yaxis.set_major_formatter(mticker.FuncFormatter(lambda x,_: f'${x/1000
```

```
plt.tight_layout(); plt.show()
```

```
print("Target Statistics:")
for lbl, val in [('Min',    train_df['TargetValue'].min()),
                 ('Max',    train_df['TargetValue'].max()),
                 ('Mean',   train_df['TargetValue'].mean()),
                 ('Median',train_df['TargetValue'].median()),
                 ('Std',    train_df['TargetValue'].std())]:
    print(f"  {lbl:8}: ${val:>12,.0f}")
print(f"\n  Skewness original : {train_df['TargetValue'].skew():.4f}")
print(f"  Skewness log1p    : {log_target.skew():.4f}  ← closer to 0 = better f
```




```
Target Statistics:
  Min      : $       7,500
  Max      : $     142,000
  Mean     : $      41,522
  Median   : $      35,000
  Std      : $      26,361

  Skewness original : 0.9700
  Skewness log1p    : -0.1983  ← closer to 0 = better for regression
```

In [12]:

```
# ═══════════════════════════════════════════════════════════
# 3.2  MISSING VALUE ANALYSIS
# ═══════════════════════════════════════════════════════════
fig, axes = plt.subplots(1, 2, figsize=(18, 8))

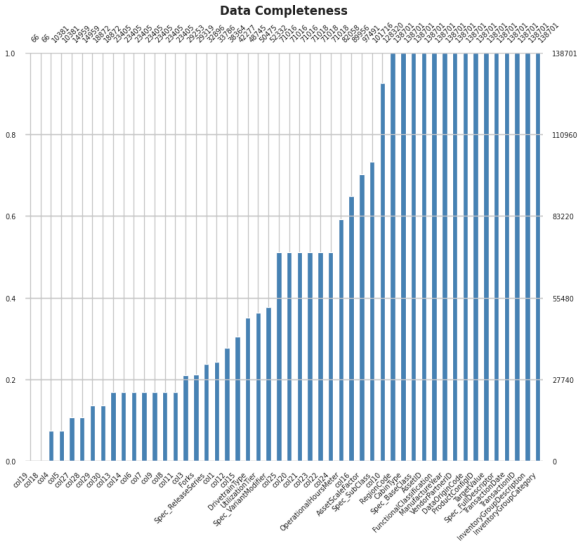
msno.bar(train_df, ax=axes[0], fontsize=7, color='steelblue', sort='ascending'
axes[0].set_title('Data Completeness', fontsize=12, fontweight='bold')

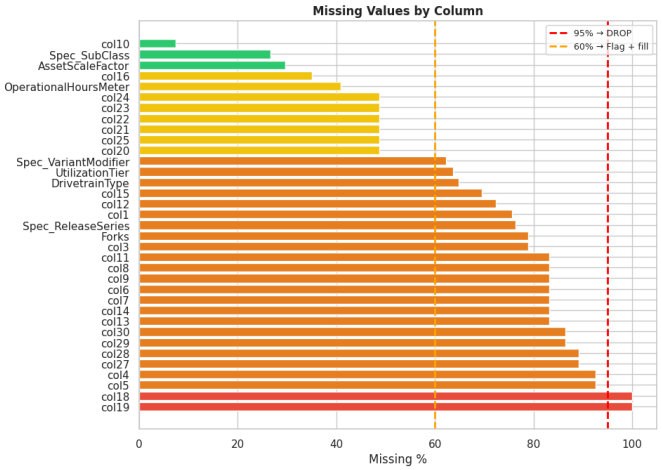
miss_pct = (train_df.isnull().sum() / len(train_df) * 100).sort_values(ascendi
miss_pct = miss_pct[miss_pct > 0]
bar_c = ['#e74c3c' if v >= 95 else '#e67e22' if v >= 60
         else '#f1c40f' if v >= 30 else '#2ecc71' for v in miss_pct]
```

```
axes[1].barh(miss_pct.index, miss_pct.values, color=bar_c, edgecolor='white')
axes[1].axvline(95, color='red',    ls='--', lw=2, label='95% → DROP')
axes[1].axvline(60, color='orange', ls='--', lw=2, label='60% → Flag + fill')
axes[1].set_xlabel('Missing %')
axes[1].set_title('Missing Values by Column', fontsize=12, fontweight='bold')
axes[1].legend(fontsize=9)

plt.tight_layout(); plt.show()

mv = pd.DataFrame({
    'Count': train_df.isnull().sum(),
    'Pct %': (train_df.isnull().sum() / len(train_df) * 100).round(2)
})
mv = mv[mv['Count'] > 0].sort_values('Pct %', ascending=False)
mv['Strategy'] = mv['Pct %'].apply(lambda x:
    'DROP' if x >= 95 else
    'Binary flag + fill "Missing"' if x >= 60 else
    'Impute median/mode')
print(mv.to_string())
```





```
                        Count  Pct %                     Strategy
col18                  138635  99.95                         DROP
col19                  138635  99.95                         DROP
col4                   128320  92.52  Binary flag + fill "Missing"
col5                   128320  92.52  Binary flag + fill "Missing"
col28                  123742  89.21  Binary flag + fill "Missing"
col27                  123742  89.21  Binary flag + fill "Missing"
col30                  119829  86.39  Binary flag + fill "Missing"
col29                  119829  86.39  Binary flag + fill "Missing"
col14                  115296  83.13  Binary flag + fill "Missing"
col11                  115296  83.13  Binary flag + fill "Missing"
col13                  115296  83.13  Binary flag + fill "Missing"
col8                   115296  83.13  Binary flag + fill "Missing"
col9                   115296  83.13  Binary flag + fill "Missing"
col6                   115296  83.13  Binary flag + fill "Missing"
col7                   115296  83.13  Binary flag + fill "Missing"
col3                   109448  78.91  Binary flag + fill "Missing"
Forks                  109382  78.86  Binary flag + fill "Missing"
Spec_ReleaseSeries     105805  76.28  Binary flag + fill "Missing"
col1                   104915  75.64  Binary flag + fill "Missing"
col12                  100337  72.34  Binary flag + fill "Missing"
col15                   96424  69.52  Binary flag + fill "Missing"
DrivetrainType          89956  64.86  Binary flag + fill "Missing"
UtilizationTier         88226  63.61  Binary flag + fill "Missing"
Spec_VariantModifier    86369  62.27  Binary flag + fill "Missing"
col24                   67683  48.80            Impute median/mode
col23                   67683  48.80            Impute median/mode
col25                   67685  48.80            Impute median/mode
col22                   67683  48.80            Impute median/mode
col21                   67685  48.80            Impute median/mode
col20                   67685  48.80            Impute median/mode
OperationalHoursMeter   56643  40.84            Impute median/mode
col16                   48745  35.14            Impute median/mode
AssetScaleFactor        41210  29.71            Impute median/mode
Spec_SubClass           36985  26.67            Impute median/mode
col10                   10381   7.48            Impute median/mode
```

In [14]:

```
# ═══════════════════════════════════════════════════════
# 3.3  CATEGORICAL FEATURES — BIVARIATE ANALYSIS
# ═══════════════════════════════════════════════════════
key_cats = ['InventoryGroupDescription','UtilizationTier','CabinType','AssetSc

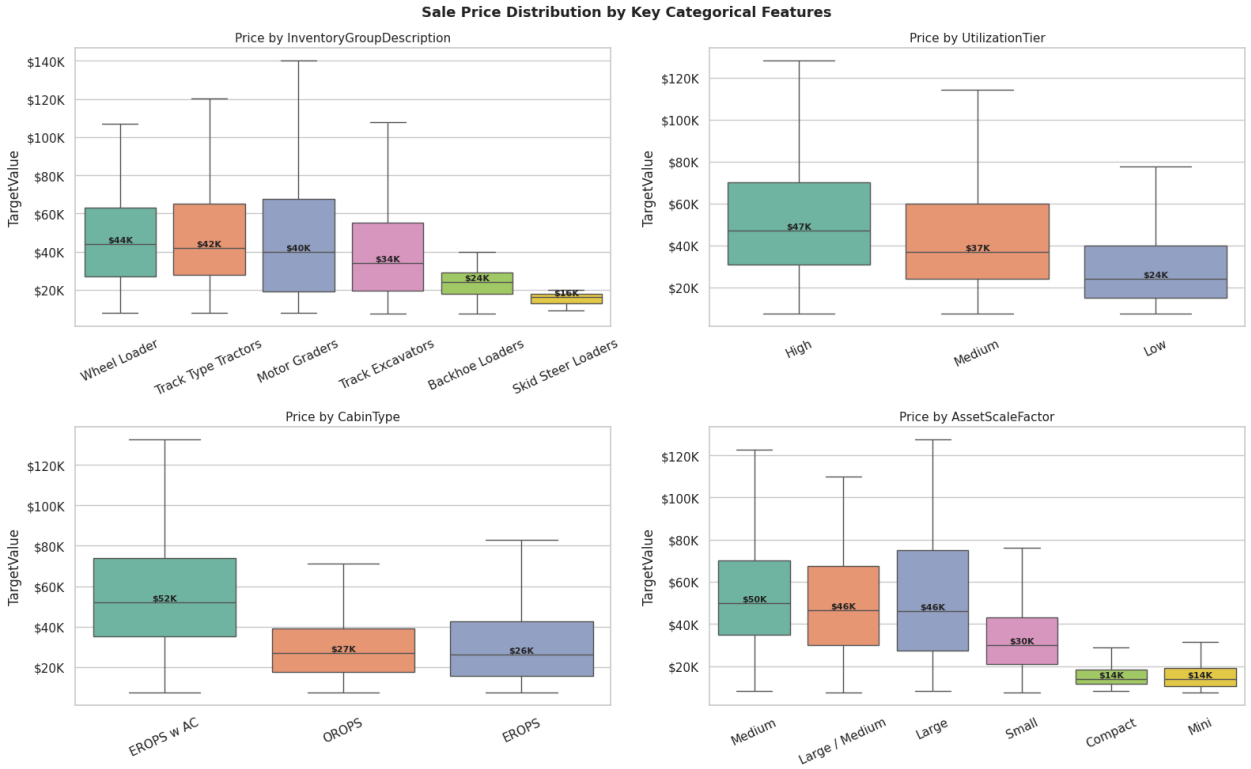
fig, axes = plt.subplots(2, 2, figsize=(16, 10))
axes = axes.flatten()
fig.suptitle('Sale Price Distribution by Key Categorical Features',
             fontsize=13, fontweight='bold')

for i, col in enumerate(key_cats):
    data  = train_df[[col,'TargetValue']].dropna()
    order = data.groupby(col)['TargetValue'].median().sort_values(ascending=Fa
    sns.boxplot(data=data, x=col, y='TargetValue', order=order,
                ax=axes[i], palette='Set2', showfliers=False)
    axes[i].set_title(f'Price by {col}', fontsize=11)
    axes[i].set_xlabel('')
```

```
    axes[i].tick_params(axis='x', rotation=25)
    axes[i].yaxis.set_major_formatter(mticker.FuncFormatter(lambda x,_: f'${x/
    for j, med in enumerate(data.groupby(col)['TargetValue'].median()[order]):
        axes[i].text(j, med, f'${med/1000:.0f}K', ha='center', va='bottom', fo

plt.tight_layout(); plt.show()

for col in key_cats:
    print(f"\n{col}:")
    print(train_df.groupby(col)['TargetValue'].agg(Count='count',Median='media
          .sort_values('Median', ascending=False).to_string())
```



```
InventoryGroupDescription:
                            Count   Median
InventoryGroupDescription
Wheel Loader                18872 44000.00
Track Type Tractors         14959 42000.00
Motor Graders               23405 40000.00
Track Excavators            71018 34000.00
Backhoe Loaders             10381 24000.00
Skid Steer Loaders             66 16250.00

UtilizationTier:
                  Count   Median
UtilizationTier
High               8882 47000.00
Medium            24588 37000.00
Low               17005 24000.00

CabinType:
             Count   Median
CabinType
EROPS w AC   55752 52000.00
OROPS        22760 27000.00
EROPS        60189 26000.00

AssetScaleFactor:
                   Count   Median
AssetScaleFactor
Medium             21918 50000.00
Large / Medium     39222 46500.00
Large               2775 46000.00
Small              18119 30000.00
Compact             1794 14000.00
Mini               13663 14000.00
```

In [15]:

```python
# ═══════════════════════════════════════════════════════════
# 3.4  NUMERICAL FEATURES
# ═══════════════════════════════════════════════════════════
fig, axes = plt.subplots(2, 2, figsize=(16, 10))
fig.suptitle('Numerical Features Analysis', fontsize=13, fontweight='bold')

# Price vs ManufactureYear
yr = train_df[train_df['ManufactureYear'].between(1975,2020)]
yr_s = yr.groupby('ManufactureYear')['TargetValue'].agg(
    M='median', Q1=lambda x: x.quantile(.25), Q3=lambda x: x.quantile(.75))
axes[0,0].plot(yr_s.index, yr_s['M'], color='steelblue', lw=2, marker='o', ms=
axes[0,0].fill_between(yr_s.index, yr_s['Q1'], yr_s['Q3'], alpha=.2, color='st
axes[0,0].set_title('Median Price by Manufacture Year (shaded=IQR)')
axes[0,0].set_xlabel('Year')
axes[0,0].yaxis.set_major_formatter(mticker.FuncFormatter(lambda x,_: f'${x/10

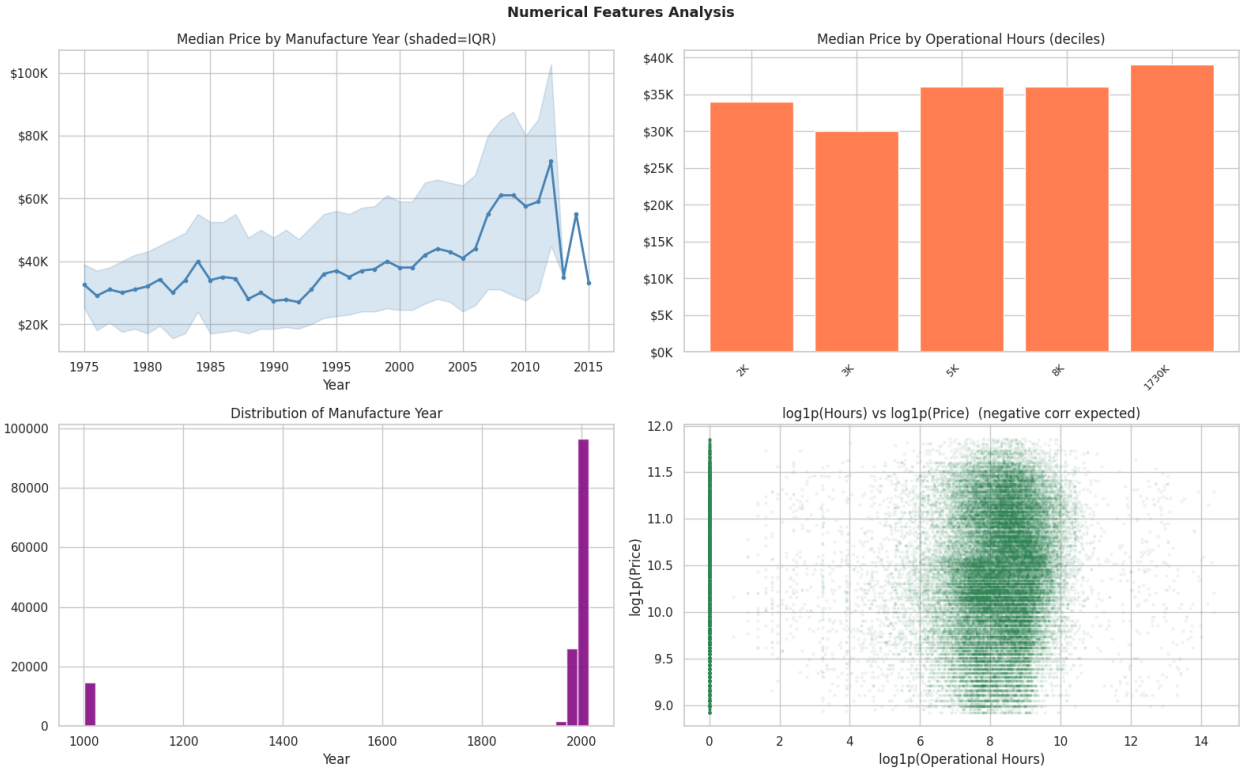
# Price vs Operational Hours (binned)
tmp = train_df[['OperationalHoursMeter','TargetValue']].dropna()
tmp['Bin'] = pd.qcut(tmp['OperationalHoursMeter'], q=8, duplicates='drop')
bp = tmp.groupby('Bin', observed=True)['TargetValue'].median()
```

```
axes[0,1].bar(range(len(bp)), bp.values, color='coral', edgecolor='white')
axes[0,1].set_xticks(range(len(bp)))
axes[0,1].set_xticklabels([f'{b.right/1000:.0f}K' for b in bp.index],
                          rotation=45, ha='right', fontsize=9)
axes[0,1].set_title('Median Price by Operational Hours (deciles)')
axes[0,1].yaxis.set_major_formatter(mticker.FuncFormatter(lambda x,_: f'${x/10

# ManufactureYear distribution
axes[1,0].hist(train_df['ManufactureYear'], bins=45, color='purple', edgecolor
axes[1,0].set_title('Distribution of Manufacture Year')
axes[1,0].set_xlabel('Year')

# Scatter: log1p(Hours) vs log1p(Price)
mask = train_df['OperationalHoursMeter'].notna()
axes[1,1].scatter(np.log1p(train_df.loc[mask,'OperationalHoursMeter']),
                  np.log1p(train_df.loc[mask,'TargetValue']),
                  alpha=.05, s=3, color='seagreen')
axes[1,1].set_title('log1p(Hours) vs log1p(Price)  (negative corr expected)')
axes[1,1].set_xlabel('log1p(Operational Hours)')
axes[1,1].set_ylabel('log1p(Price)')

plt.tight_layout(); plt.show()
print("Key insight: Newer machines → higher price | More hours → lower price (
```




```
Key insight: Newer machines → higher price | More hours → lower price (deprecia
tion)
```

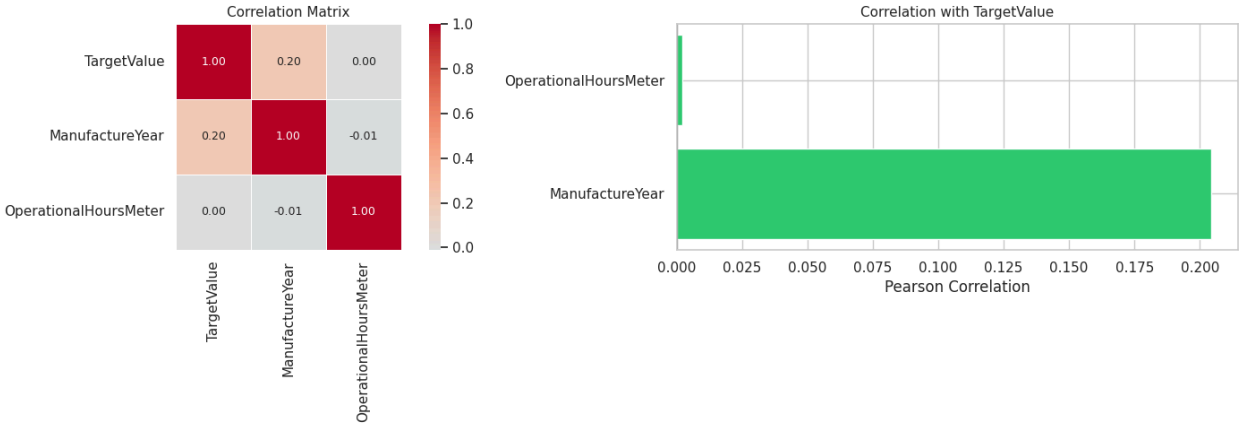
In [16]:
```
# ═══════════════════════════════════════════════════════════
# 3.5  CORRELATION ANALYSIS
# ═══════════════════════════════════════════════════════════
num_cols = [c for c in train_df.select_dtypes(include=[np.number]).columns
```

```
            if c not in ['TransactionID','AssetID','ProductConfigID']]
corr = train_df[num_cols].corr()
tgt_corr = corr['TargetValue'].drop('TargetValue').sort_values(key=abs, ascend

fig, axes = plt.subplots(1, 2, figsize=(16, 5))
sns.heatmap(corr, ax=axes[0], annot=True, fmt='.2f', cmap='coolwarm',
            center=0, square=True, linewidths=.5, annot_kws={'size':9})
axes[0].set_title('Correlation Matrix', fontsize=11)

colors = ['#2ecc71' if v>=0 else '#e74c3c' for v in tgt_corr]
axes[1].barh(tgt_corr.index, tgt_corr.values, color=colors, edgecolor='white')
axes[1].axvline(0, color='black', lw=.8)
axes[1].set_xlabel('Pearson Correlation')
axes[1].set_title('Correlation with TargetValue', fontsize=11)
plt.tight_layout(); plt.show()

print("Correlations with TargetValue:")
print(tgt_corr.to_string())
```




```
Correlations with TargetValue:
ManufactureYear         0.20
OperationalHoursMeter   0.00
```

In [17]:

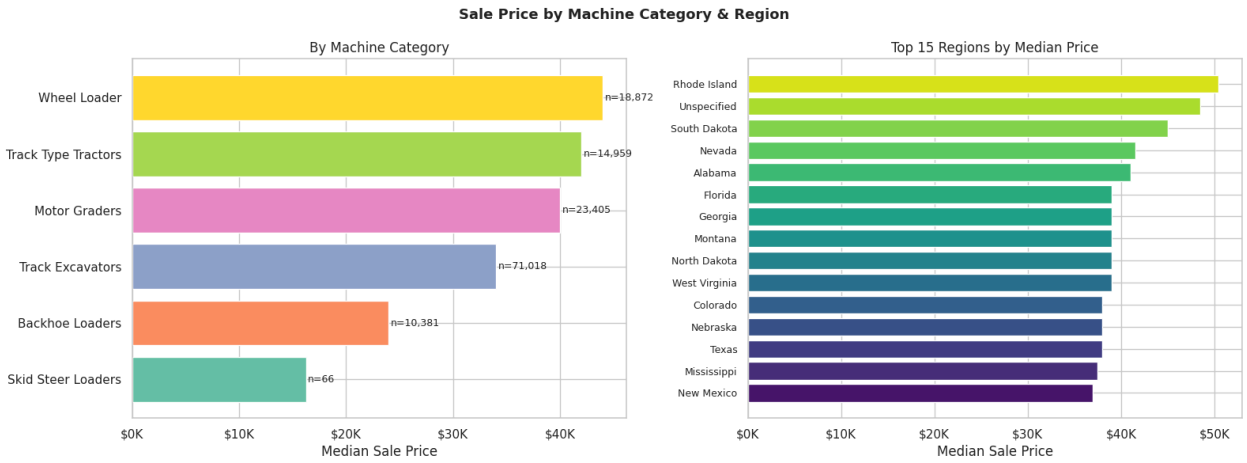
```
# ═══════════════════════════════════════════════════════════
# 3.6  PRICE BY MACHINE TYPE & REGION
# ═══════════════════════════════════════════════════════════
fig, axes = plt.subplots(1, 2, figsize=(16, 6))
fig.suptitle('Sale Price by Machine Category & Region', fontsize=13, fontweigh

ig = (train_df.groupby('InventoryGroupDescription')['TargetValue']
      .agg(Median='median', Count='count').sort_values('Median').reset_index()
bars = axes[0].barh(ig['InventoryGroupDescription'], ig['Median'],
                    color=sns.color_palette('Set2', len(ig)), edgecolor='whit
axes[0].set_xlabel('Median Sale Price')
axes[0].set_title('By Machine Category')
axes[0].xaxis.set_major_formatter(mticker.FuncFormatter(lambda x,_: f'${x/1000
for bar, cnt in zip(bars, ig['Count']):
    axes[0].text(bar.get_width()+200, bar.get_y()+bar.get_height()/2,
                 f'n={cnt:,}', va='center', fontsize=9)

reg = train_df.groupby('RegionCode')['TargetValue'].median().nlargest(15)
```

```
axes[1].barh(range(len(reg)), reg.values[::-1],
             color=sns.color_palette('viridis', len(reg)), edgecolor='white')
axes[1].set_yticks(range(len(reg)))
axes[1].set_yticklabels(reg.index[::-1], fontsize=9)
axes[1].set_xlabel('Median Sale Price')
axes[1].set_title('Top 15 Regions by Median Price')
axes[1].xaxis.set_major_formatter(mticker.FuncFormatter(lambda x,_: f'${x/1000
```

```
plt.tight_layout(); plt.show()
```



4. Data Preprocessing & Feature Engineering

Strategy

Step	Action
Drop	`col18`, `col19` (99.95% missing), `TransactionID`, `AssetID`
Date extract	`SaleYear`, `SaleMonth`, `SaleQuarter` from `TransactionDate`
MachineAge ⭐	`SaleYear − ManufactureYear` — strongest depreciation signal
UsagePerYear	`OperationalHoursMeter / (MachineAge+1)`
Missing flags	Binary `_present` for high-missing columns
Imputation	Median (numeric) / `'Missing'` (categorical)
Encoding	`OrdinalEncoder` for all categoricals
Scaling	`StandardScaler` for all numerics

> All transformations use **sklearn Pipeline + ColumnTransformer** — fitted only on training data to prevent leakage.

In [18]:
```
# ═══════════════════════════════════════════════════════
# 4.1  FEATURE ENGINEERING
```

```python
# Applied IDENTICALLY to train and test — no leakage
# ═══════════════════════════════════════════════════
def engineer_features(df):  # Create meaningful new features from raw columns
    df = df.copy()

    # Date decomposition
    df['TransactionDate'] = pd.to_datetime(df['TransactionDate'], errors='coer
    df['SaleYear']    = df['TransactionDate'].dt.year
    df['SaleMonth']   = df['TransactionDate'].dt.month
    df['SaleQuarter'] = df['TransactionDate'].dt.quarter
    df.drop(columns=['TransactionDate'], inplace=True)

    # MachineAge — KEY feature: older machine = lower price
    df['MachineAge'] = (df['SaleYear'] - df['ManufactureYear']).clip(lower=0)

    # Usage intensity: high hours/year → faster depreciation
    df['UsagePerYear'] = df['OperationalHoursMeter'] / (df['MachineAge'] + 1)

    # Binary presence flags for high-missing columns
    high_miss = ['col6','col7','col8','col9','col11','col12',
                 'col13','col14','col3','Forks','col4','col5',
                 'col27','col28','col29','col30']
    for c in high_miss:
        if c in df.columns:
            df[f'{c}_present'] = df[c].notna().astype(np.int8)

    # Standardise "None or Unspecified" → NaN
    obj = df.select_dtypes(include='object').columns
    df[obj] = df[obj].replace('None or Unspecified', np.nan)

    # Drop near-empty columns
    df.drop(columns=[c for c in ['col18','col19'] if c in df.columns], inplace

    return df

train_eng = engineer_features(train_df)
test_eng  = engineer_features(test_df)

print("Feature engineering applied!")
print(f"  Train: {train_df.shape} → {train_eng.shape}")
print(f"  Test : {test_df.shape}  → {test_eng.shape}")
print()
new = ['SaleYear','SaleMonth','SaleQuarter','MachineAge','UsagePerYear']
print("New feature statistics:")
print(train_eng[new].describe().round(2).to_string())
```

```
Feature engineering applied!
  Train: (138701, 50) → (138701, 68)
  Test : (15000, 49)  → (15000, 67)

New feature statistics:
       SaleYear  SaleMonth  SaleQuarter  MachineAge  UsagePerYear
count 138701.00  138701.00    138701.00   138701.00      82058.00
mean    2007.49       6.26         2.34      115.85        464.32
std        4.79       3.45         1.14      307.18       3118.53
min     1990.00       1.00         1.00        0.00          0.00
25%     2005.00       3.00         1.00        5.00          0.00
50%     2009.00       6.00         2.00        9.00        187.76
75%     2011.00       9.00         3.00       16.00        551.57
max     2013.00      12.00         4.00     1012.00     200362.50
```

In [19]:

```python
# ═══════════════════════════════════════════════════════
# 4.2  PREPARE FEATURE MATRICES & TARGET
# ═══════════════════════════════════════════════════════
ID_COLS = ['TransactionID', 'AssetID']

# log1p(price) → symmetric distribution, aligns with RMSLE optimisation
y_all  = np.log1p(train_eng['TargetValue'])
X_all  = train_eng.drop(columns=ID_COLS + ['TargetValue'], errors='ignore')
X_test_final = test_eng.drop(columns=ID_COLS, errors='ignore')
test_ids     = test_df['TransactionID'].values

print(f"X_all   : {X_all.shape}")
print(f"y_all   : {y_all.shape}  (log1p scale)")
print(f"X_test  : {X_test_final.shape}")

# 80 / 20 train-validation split
X_train, X_val, y_train, y_val = train_test_split(
    X_all, y_all, test_size=0.2, random_state=SEED)

print(f"\nSplit 80/20:")
print(f"  X_train: {X_train.shape}   X_val: {X_val.shape}")
print(f"\nTarget (log1p) range:")
print(f"  Min {y_all.min():.3f} → ${np.expm1(y_all.min()):,.0f}")
print(f"  Max {y_all.max():.3f} → ${np.expm1(y_all.max()):,.0f}")
```

```
X_all   : (138701, 65)
y_all   : (138701,)  (log1p scale)
X_test  : (15000, 65)

Split 80/20:
  X_train: (110960, 65)   X_val: (27741, 65)

Target (log1p) range:
  Min 8.923 → $7,500
  Max 11.864 → $142,000
```

In [20]:

```python
# ═══════════════════════════════════════════════════════
# 4.3  SKLEARN PREPROCESSING PIPELINE
# ═══════════════════════════════════════════════════════
```

```
numeric_cols = X_train.select_dtypes(include=[np.number]).columns.tolist()
cat_cols     = X_train.select_dtypes(include=['object']).columns.tolist()

print(f"Numeric  cols ({len(numeric_cols)}): {numeric_cols}")
print(f"\nCategorical cols ({len(cat_cols)}): {cat_cols}")

numeric_transformer = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler',  StandardScaler())
])

categorical_transformer = Pipeline([
    ('imputer', SimpleImputer(strategy='constant', fill_value='Missing')),
    ('encoder', OrdinalEncoder(handle_unknown='use_encoded_value', unknown_val
])

preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_cols),
        ('cat', categorical_transformer, cat_cols),
    ],
    remainder='drop',
    verbose_feature_names_out=False
)

# fit_transform only on TRAIN — transform val/test without refitting
X_train_proc = preprocessor.fit_transform(X_train)
X_val_proc   = preprocessor.transform(X_val)
X_test_proc  = preprocessor.transform(X_test_final)

feature_names_out = numeric_cols + cat_cols

print(f"\nPreprocessed shapes:")
print(f"  X_train_proc : {X_train_proc.shape}")
print(f"  X_val_proc   : {X_val_proc.shape}")
print(f"  X_test_proc  : {X_test_proc.shape}")
print("\nPipeline summary:")
print("  Numeric  → Median Imputation → StandardScaler")
print("  Categorical → fill 'Missing' → OrdinalEncoder")
```

```
Numeric  cols (24): ['ProductConfigID', 'ManufactureYear', 'OperationalHoursMet
er', 'SaleYear', 'SaleMonth', 'SaleQuarter', 'MachineAge', 'UsagePerYear', 'col
6_present', 'col7_present', 'col8_present', 'col9_present', 'col11_present', 'c
ol12_present', 'col13_present', 'col14_present', 'col3_present', 'Forks_presen
t', 'col4_present', 'col5_present', 'col27_present', 'col28_present', 'col29_pr
esent', 'col30_present']

Categorical cols (41): ['DataOriginCode', 'VendorPartnerID', 'UtilizationTier',
'Spec_FullDescriptor', 'Spec_BaseClass', 'Spec_SubClass', 'Spec_ReleaseSeries',
'Spec_VariantModifier', 'AssetScaleFactor', 'FunctionalClassification', 'Region
Code', 'InventoryGroupCategory', 'InventoryGroupDescription', 'col1', 'CabinTyp
e', 'Forks', 'col3', 'col4', 'DrivetrainType', 'col5', 'col6', 'col7', 'col8',
'col9', 'col10', 'col11', 'col12', 'col13', 'col14', 'col15', 'col16', 'col20',
'col21', 'col22', 'col23', 'col24', 'col25', 'col27', 'col28', 'col29', 'col3
0']

Preprocessed shapes:
  X_train_proc : (110960, 65)
  X_val_proc   : (27741, 65)
  X_test_proc  : (15000, 65)

Pipeline summary:
  Numeric  → Median Imputation → StandardScaler
  Categorical → fill 'Missing' → OrdinalEncoder
```

5. Model Building & Comparison

Five models from **different ML families** are compared on the 20 % validation set:

#	Model	Family	Strength	Weakness
0	DummyRegressor	Baseline	—	Always predicts mean
1	Ridge Regression	Linear	Fast, interpretable	Linear only
2	Random Forest	Bagging Ensemble	Robust, non-linear	Memory-heavy
3	HistGradientBoosting	Boosting Ensemble	Fast, handles NaN	Fewer options
4	XGBoost	Boosting Ensemble	Best accuracy, L1+L2	Many params
5	LightGBM	Boosting Ensemble	Fastest, large-data	Can overfit

In [23]:
```python
# ── RMSLE — competition metric ──────────────────────────
def rmsle(y_true, y_pred):
    # Root Mean Squared Logarithmic Error. Lower = better. Target <= 0.85.
    y_pred = np.maximum(y_pred, 0)
    return np.sqrt(mean_squared_error(np.log1p(y_true), np.log1p(y_pred)))
```

```
def neg_rmsle_log(y_true, y_pred):
    # Scorer for sklearn when target is already log1p transformed.
    return -np.sqrt(mean_squared_error(y_true, y_pred))

rmsle_scorer = make_scorer(neg_rmsle_log)

# ── Submission generator ──────────────────────────────────
def make_submission(model, X_proc, test_ids, log_target=True,
                    filename='submission.csv'):
    preds = model.predict(X_proc)
    if log_target:
        preds = np.expm1(preds)
    preds = np.maximum(preds, 0)
    sub = pd.DataFrame({'TransactionID': test_ids, 'TargetValue': preds})
    sub.to_csv(filename, index=False)
    print(f"✅ {filename} saved ({len(sub):,} rows)")
    return sub

print("\n✅ Helpers ready.")
```

```
✅ Helpers ready.
```

In [26]:

```
# ═══════════════════════════════════════════════════════
# 5.  MODEL BUILDING & COMPARISON
# ═══════════════════════════════════════════════════════
model_results  = {}
trained_models = {}

print("=" * 60)
print("  MODEL COMPARISON — RMSLE on 20% Validation Set")
print("=" * 60)

# ── Baseline: DummyRegressor ──────────────────────────────
dummy = DummyRegressor(strategy='mean')
dummy.fit(X_train_proc, y_train)
d_r = rmsle(np.expm1(y_val), np.expm1(dummy.predict(X_val_proc)))
model_results['0. Dummy Baseline']   = d_r
trained_models['0. Dummy Baseline']  = dummy
print(f"\n  0. DummyRegressor (mean)         RMSLE: {d_r:.4f}  [must beat this

# ── Model 1: Ridge Regression (Linear) ────────────────────
# Pros: Extremely fast, interpretable coefficients
# Cons: Captures linear relationships only — will underfit this data
ridge = Ridge(alpha=10.0, random_state=SEED)
ridge.fit(X_train_proc, y_train)
r_r = rmsle(np.expm1(y_val), np.expm1(ridge.predict(X_val_proc)))
model_results['1. Ridge Regression']  = r_r
trained_models['1. Ridge Regression'] = ridge
print(f"  1. Ridge Regression              RMSLE: {r_r:.4f}")
print(f"     Pros: Fast, interpretable | Cons: Linear only")
```

```
============================================================
  MODEL COMPARISON — RMSLE on 20% Validation Set
============================================================

  0. DummyRegressor (mean)         RMSLE: 0.6694  [must beat this]
  1. Ridge Regression              RMSLE: 0.5296
     Pros: Fast, interpretable | Cons: Linear only
```

In [27]:
```
# ── Model 2: Random Forest (Bagging) ──────────────────
# Pros: Handles non-linearity, robust to outliers, good feature importance
# Cons: Slower than boosting, memory-intensive with many trees
rf = RandomForestRegressor(n_estimators=200, max_depth=12, min_samples_leaf=5,
                           max_features='sqrt', random_state=SEED, n_jobs=-1)
rf.fit(X_train_proc, y_train)
rf_r = rmsle(np.expm1(y_val), np.expm1(rf.predict(X_val_proc)))
model_results['2. Random Forest']  = rf_r
trained_models['2. Random Forest'] = rf
print(f"  2. Random Forest                 RMSLE: {rf_r:.4f}")
print(f"     Pros: Non-linear, robust | Cons: Slower, memory-heavy")

# ── Model 3: HistGradientBoosting (Boosting) ──────────
# Pros: Fast, handles NaN natively, built-in regularisation
# Cons: Fewer tuning knobs than XGBoost / LightGBM
hgb = HistGradientBoostingRegressor(max_iter=500, max_depth=6, learning_rate=0
                                    min_samples_leaf=20, l2_regularization=1.
                                    random_state=SEED)
hgb.fit(X_train_proc, y_train)
hgb_r = rmsle(np.expm1(y_val), np.expm1(hgb.predict(X_val_proc)))
model_results['3. HistGradientBoosting']  = hgb_r
trained_models['3. HistGradientBoosting'] = hgb
print(f"  3. HistGradientBoosting          RMSLE: {hgb_r:.4f}")
print(f"     Pros: Fast, handles NaN | Cons: Fewer tuning options")
```

```
  2. Random Forest                 RMSLE: 0.2895
     Pros: Non-linear, robust | Cons: Slower, memory-heavy
  3. HistGradientBoosting          RMSLE: 0.2327
     Pros: Fast, handles NaN | Cons: Fewer tuning options
```

In [28]:
```
# ── Model 4: XGBoost (Boosting) ───────────────────────
# Pros: Outstanding accuracy, L1+L2 regularisation, sparse data friendly
# Cons: Many hyperparameters, slower than LightGBM
xgb_m = XGBRegressor(n_estimators=700, max_depth=6, learning_rate=0.05,
                     subsample=0.8, colsample_bytree=0.8,
                     reg_alpha=0.1, reg_lambda=1.0, min_child_weight=3,
                     tree_method='hist', random_state=SEED, n_jobs=-1, verbos
xgb_m.fit(X_train_proc, y_train, eval_set=[(X_val_proc, y_val)], verbose=False
xgb_r = rmsle(np.expm1(y_val), np.expm1(xgb_m.predict(X_val_proc)))
model_results['4. XGBoost']  = xgb_r
trained_models['4. XGBoost'] = xgb_m
print(f"  4. XGBoost                       RMSLE: {xgb_r:.4f}")
print(f"     Pros: Best accuracy, L1+L2 reg | Cons: Many params, slower")

# ── Model 5: LightGBM (Boosting) ──────────────────────
# Pros: Fastest boosting, great with 100K+ rows, leaf-wise growth
```

```
# Cons: Can overfit on small datasets, sensitive to hyperparameters
lgbm_m = lgb.LGBMRegressor(n_estimators=700, max_depth=6, learning_rate=0.05,
                           subsample=0.8, colsample_bytree=0.8,
                           reg_alpha=0.1, reg_lambda=1.0,
                           min_child_samples=20, num_leaves=63,
                           random_state=SEED, n_jobs=-1, verbose=-1)
lgbm_m.fit(X_train_proc, y_train, eval_set=[(X_val_proc, y_val)],
           callbacks=[lgb.early_stopping(50, verbose=False), lgb.log_evaluatic
lgbm_r = rmsle(np.expm1(y_val), np.expm1(lgbm_m.predict(X_val_proc)))
model_results['5. LightGBM']  = lgbm_r
trained_models['5. LightGBM'] = lgbm_m
print(f"  5. LightGBM                      RMSLE: {lgbm_r:.4f}")
print(f"     Pros: Fastest, great at scale | Cons: Can overfit, sensitive para
print(f"\n{'='*60}")
```

```
 4. XGBoost                       RMSLE: 0.2208
    Pros: Best accuracy, L1+L2 reg | Cons: Many params, slower
 5. LightGBM                      RMSLE: 0.2217
    Pros: Fastest, great at scale | Cons: Can overfit, sensitive params

============================================================
```

In [29]:

```
# ── Model Comparison Chart ──────────────────────────
fig, ax = plt.subplots(figsize=(12, 6))

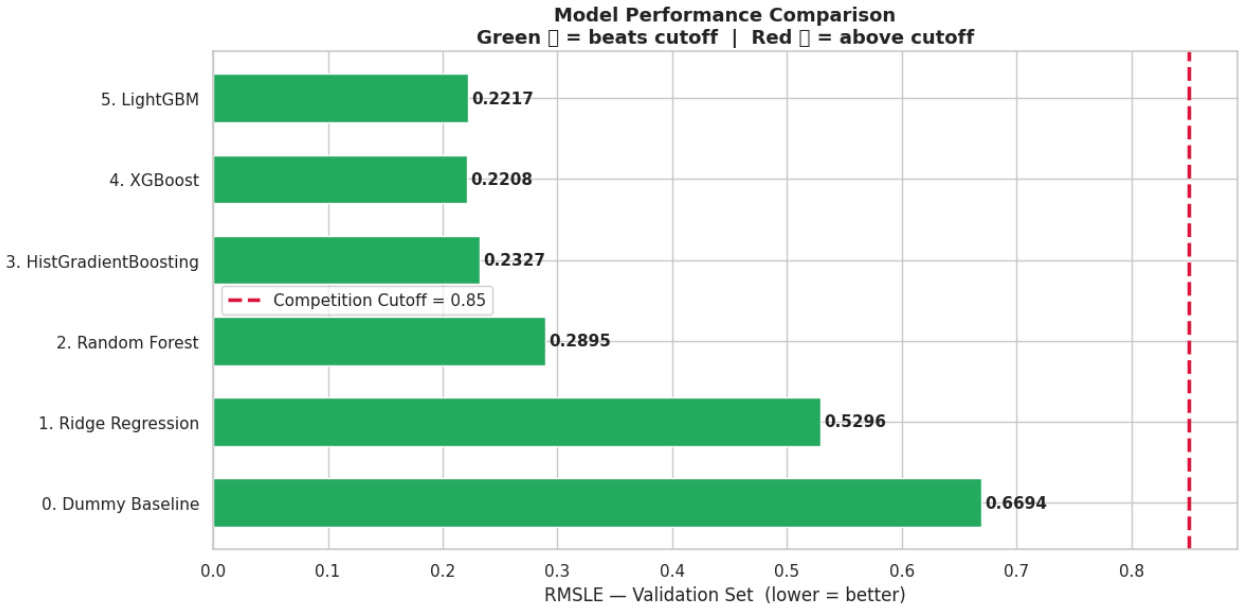
names  = list(model_results.keys())
scores = list(model_results.values())
colors = ['#27ae60' if s < 0.85 else '#e74c3c' for s in scores]

bars = ax.barh(names, scores, color=colors, edgecolor='white', height=0.6)
ax.axvline(0.85, color='crimson', ls='--', lw=2.5, label='Competition Cutoff =
ax.set_xlabel('RMSLE — Validation Set  (lower = better)')
ax.set_title('Model Performance Comparison\n'
             'Green ✅ = beats cutoff  |  Red ❌ = above cutoff',
             fontsize=13, fontweight='bold')
ax.legend(fontsize=11)
for bar, sc in zip(bars, scores):
    ax.text(sc+0.003, bar.get_y()+bar.get_height()/2,
            f'{sc:.4f}', va='center', fontsize=11, fontweight='bold')

plt.tight_layout(); plt.show()

print("\nFull comparison:")
print(f"{'Model':<35} {'RMSLE':>10}  {'Gap to 0.85':>12}  Status")
print("-"*72)
for name, sc in sorted(model_results.items(), key=lambda x: x[1]):
    gap = 0.85 - sc
    st  = '✅ BEATS CUTOFF' if sc < 0.85 else '❌ Above cutoff'
    print(f"{name:<35} {sc:>10.4f}  {gap:>+12.4f}  {st}")

best = min(model_results, key=model_results.get)
print(f"\n Best model: {best}  RMSLE={model_results[best]:.4f}")
```




```
Full comparison:
Model                                    RMSLE   Gap to 0.85  Status
------------------------------------------------------------------------
4. XGBoost                              0.2208       +0.6292  ✅ BEATS CUTOFF
5. LightGBM                             0.2217       +0.6283  ✅ BEATS CUTOFF
3. HistGradientBoosting                 0.2327       +0.6173  ✅ BEATS CUTOFF
2. Random Forest                        0.2895       +0.5605  ✅ BEATS CUTOFF
1. Ridge Regression                     0.5296       +0.3204  ✅ BEATS CUTOFF
0. Dummy Baseline                       0.6694       +0.1806  ✅ BEATS CUTOFF

 Best model: 4. XGBoost  RMSLE=0.2208
```

6. Hyperparameter Tuning

RandomizedSearchCV is used on XGBoost (typically the best performer):

- Samples randomly from a parameter distribution — wider exploration, fewer evaluations
- 3-fold CV for speed (increase `cv=5` for a more reliable estimate)
- Custom `rmsle_scorer` passed directly — sklearn maximises, we negate RMSLE

Increase `n_iter` from **25 → 50+** for a more thorough search at the cost of runtime.

In [30]:
```
# ═══════════════════════════════════════════════════════════
# 6.  HYPERPARAMETER TUNING — RandomizedSearchCV (XGBoost)
# ═══════════════════════════════════════════════════════════
print("Running RandomizedSearchCV (25 iterations × 3-fold CV) ...")

param_dist = {
    'n_estimators':     [300, 500, 700, 1000],
```

```
    'max_depth':        [4, 5, 6, 7, 8],
    'learning_rate':    [0.01, 0.03, 0.05, 0.08, 0.1],
    'subsample':        [0.7, 0.8, 0.9, 1.0],
    'colsample_bytree': [0.7, 0.8, 0.9, 1.0],
    'reg_alpha':        [0, 0.05, 0.1, 0.3, 0.5],
    'reg_lambda':       [0.5, 1.0, 2.0, 5.0],
    'min_child_weight': [1, 3, 5, 7, 10],
    'gamma':            [0, 0.1, 0.3, 0.5],
}

rs = RandomizedSearchCV(
    XGBRegressor(tree_method='hist', random_state=SEED, n_jobs=-1, verbosity=0
    param_distributions=param_dist,
    n_iter=25,          # ← increase to 50-100 for better results
    scoring=rmsle_scorer,
    cv=3,
    random_state=SEED,
    n_jobs=-1,
    verbose=1,
    refit=True
)
rs.fit(X_train_proc, y_train)

print("\nBest Parameters:")
for p, v in sorted(rs.best_params_.items()):
    print(f"  {p:<25}: {v}")

tuned_preds = rs.best_estimator_.predict(X_val_proc)
tuned_rmsle = rmsle(np.expm1(y_val), np.expm1(tuned_preds))
print(f"\n  CV RMSLE (log space)  : {-rs.best_score_:.4f}")
print(f"  Val RMSLE (default)   : {xgb_r:.4f}")
print(f"  Val RMSLE (tuned)     : {tuned_rmsle:.4f}")
print(f"  Improvement           : {xgb_r - tuned_rmsle:+.4f}")
```

```
Running RandomizedSearchCV (25 iterations × 3-fold CV) ...
Fitting 3 folds for each of 25 candidates, totalling 75 fits

Best Parameters:
  colsample_bytree         : 0.8
  gamma                    : 0
  learning_rate            : 0.08
  max_depth                : 8
  min_child_weight         : 5
  n_estimators             : 1000
  reg_alpha                : 0.3
  reg_lambda               : 1.0
  subsample                : 1.0

  CV RMSLE (log space)  : 0.2120
  Val RMSLE (default)   : 0.2208
  Val RMSLE (tuned)     : 0.2045
  Improvement           : +0.0163
```

In [31]:
```
# ── Final Model: train on ALL data (train + val) ──────
```

```
print("Training final model on 100% of training data ...")

preprocessor_final = ColumnTransformer(
    transformers=[
        ('num', Pipeline([('i', SimpleImputer(strategy='median')),
                          ('s', StandardScaler())]), numeric_cols),
        ('cat', Pipeline([('i', SimpleImputer(strategy='constant', fill_value=
                          ('e', OrdinalEncoder(handle_unknown='use_encoded_val
                                               unknown_value=-1))]), cat_cols)
    ], remainder='drop', verbose_feature_names_out=False
)
X_all_proc        = preprocessor_final.fit_transform(X_all)
X_test_final_proc = preprocessor_final.transform(X_test_final)

final_model = XGBRegressor(**rs.best_params_, tree_method='hist',
                           random_state=SEED, n_jobs=-1, verbosity=0)
final_model.fit(X_all_proc, y_all)

print(f"\nFinal model trained on {X_all_proc.shape[0]:,} samples, {X_all_proc.
print(f"\n{'='*55}")
print(f"  PERFORMANCE SUMMARY")
print(f"{'='*55}")
for nm, sc in sorted(model_results.items(), key=lambda x: x[1]):
    print(f"  {nm:<35} {sc:.4f}")
print(f"  {'XGBoost (tuned HPT)':<35} {tuned_rmsle:.4f}  ← Final")
print(f"  {'Competition Cutoff':<35} 0.8500")
st = 'BEATS CUTOFF ✅' if tuned_rmsle < 0.85 else 'Needs improvement ⚠'
print(f"  Status: {st}")
```

```
Training final model on 100% of training data ...

Final model trained on 138,701 samples, 65 features

=======================================================
  PERFORMANCE SUMMARY
=======================================================
  4. XGBoost                          0.2208
  5. LightGBM                         0.2217
  3. HistGradientBoosting             0.2327
  2. Random Forest                    0.2895
  1. Ridge Regression                 0.5296
  0. Dummy Baseline                   0.6694
  XGBoost (tuned HPT)                 0.2045  ← Final
  Competition Cutoff                  0.8500
  Status: BEATS CUTOFF ✅
```

7. Error & Residual Analysis

Understanding *where* the model is wrong guides further improvement:

- **Actual vs Predicted** — perfect model sits on the diagonal
- **Residuals distribution** — centred at 0 = unbiased

- **Residuals vs Predicted** — horizontal band = homoscedastic (good)
- **Feature importance** — what drives price predictions

In [32]:
```
# ═══════════════════════════════════════════════════════
# 7.1  RESIDUAL ANALYSIS
# ═══════════════════════════════════════════════════════
pred_log   = rs.best_estimator_.predict(X_val_proc)
pred_price = np.expm1(pred_log)
true_price = np.expm1(y_val.values)

resid    = true_price - pred_price
pct_err  = (resid / true_price) * 100

fig, axes = plt.subplots(2, 2, figsize=(16, 10))
fig.suptitle('Error & Residual Analysis — Validation Set', fontsize=14, fontwe

# Actual vs Predicted
axes[0,0].scatter(true_price, pred_price, alpha=.25, s=5, color='steelblue')
mn, mx = true_price.min(), true_price.max()
axes[0,0].plot([mn,mx],[mn,mx], 'r--', lw=2, label='Perfect')
axes[0,0].set_xlabel('Actual Price'); axes[0,0].set_ylabel('Predicted Price')
axes[0,0].set_title('Actual vs Predicted')
axes[0,0].legend()
axes[0,0].xaxis.set_major_formatter(mticker.FuncFormatter(lambda x,_: f'${x/10
axes[0,0].yaxis.set_major_formatter(mticker.FuncFormatter(lambda x,_: f'${x/10

# Residuals distribution
axes[0,1].hist(resid, bins=80, color='coral', edgecolor='white', alpha=.85)
axes[0,1].axvline(0, color='black', lw=2, ls='--', label='Zero error')
axes[0,1].axvline(resid.mean(), color='red', lw=2, label=f'Mean={resid.mean():
axes[0,1].set_xlabel('Residual (Actual − Predicted)'); axes[0,1].legend(fontsi
axes[0,1].set_title('Residuals Distribution')
axes[0,1].xaxis.set_major_formatter(mticker.FuncFormatter(lambda x,_: f'${x/10

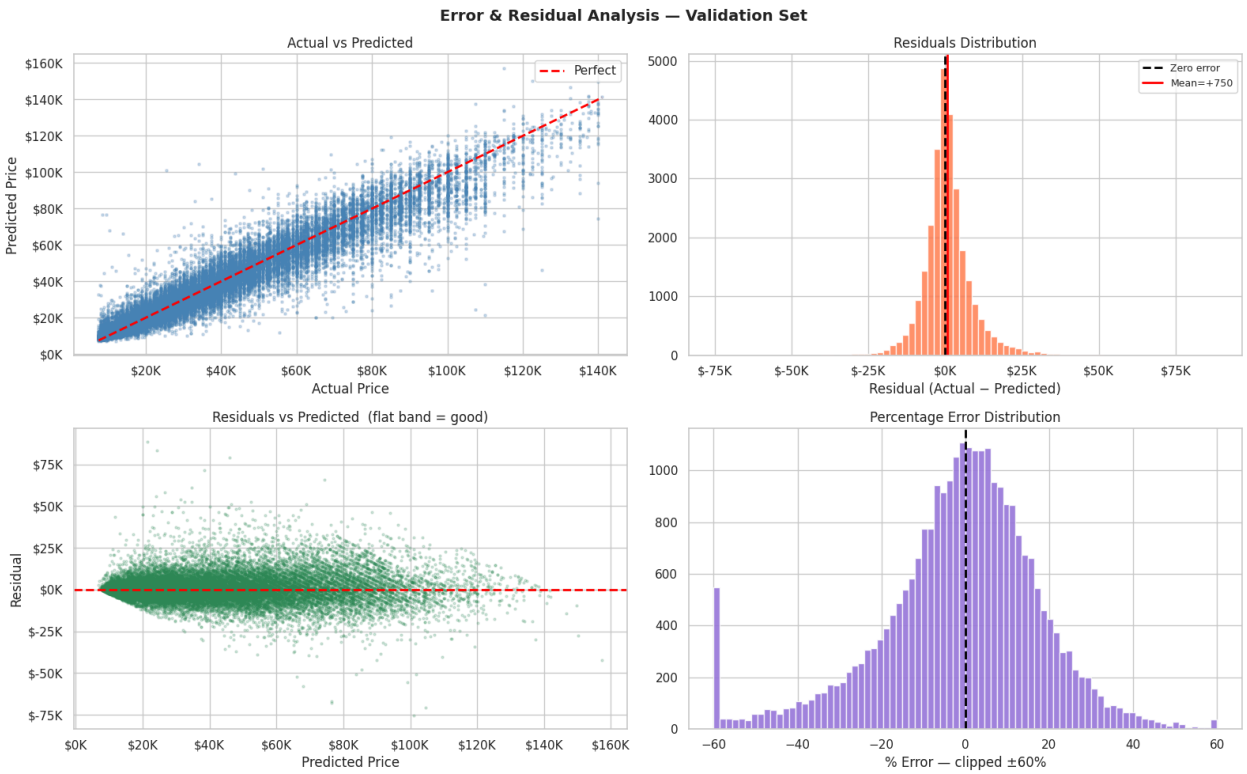
# Residuals vs Predicted
axes[1,0].scatter(pred_price, resid, alpha=.2, s=4, color='seagreen')
axes[1,0].axhline(0, color='red', lw=2, ls='--')
axes[1,0].set_xlabel('Predicted Price'); axes[1,0].set_ylabel('Residual')
axes[1,0].set_title('Residuals vs Predicted  (flat band = good)')
axes[1,0].xaxis.set_major_formatter(mticker.FuncFormatter(lambda x,_: f'${x/10
axes[1,0].yaxis.set_major_formatter(mticker.FuncFormatter(lambda x,_: f'${x/10

# % error distribution
axes[1,1].hist(pct_err.clip(-60, 60), bins=80, color='mediumpurple', edgecolor
axes[1,1].axvline(0, color='black', lw=2, ls='--')
axes[1,1].set_xlabel('% Error — clipped ±60%')
axes[1,1].set_title('Percentage Error Distribution')

plt.tight_layout(); plt.show()

print("Error Metrics — Validation Set:")
print(f"  RMSLE  : {tuned_rmsle:.4f}  (competition metric)")
```

```
print(f"  MAE    : ${np.mean(np.abs(resid)):>10,.2f}")
print(f"  RMSE   : ${np.sqrt(np.mean(resid**2)):>10,.2f}")
print(f"  R2     :  {r2_score(true_price, pred_price):>10.4f}")
print(f"  MedAPE :  {np.median(np.abs(pct_err)):>10.2f}%")
```



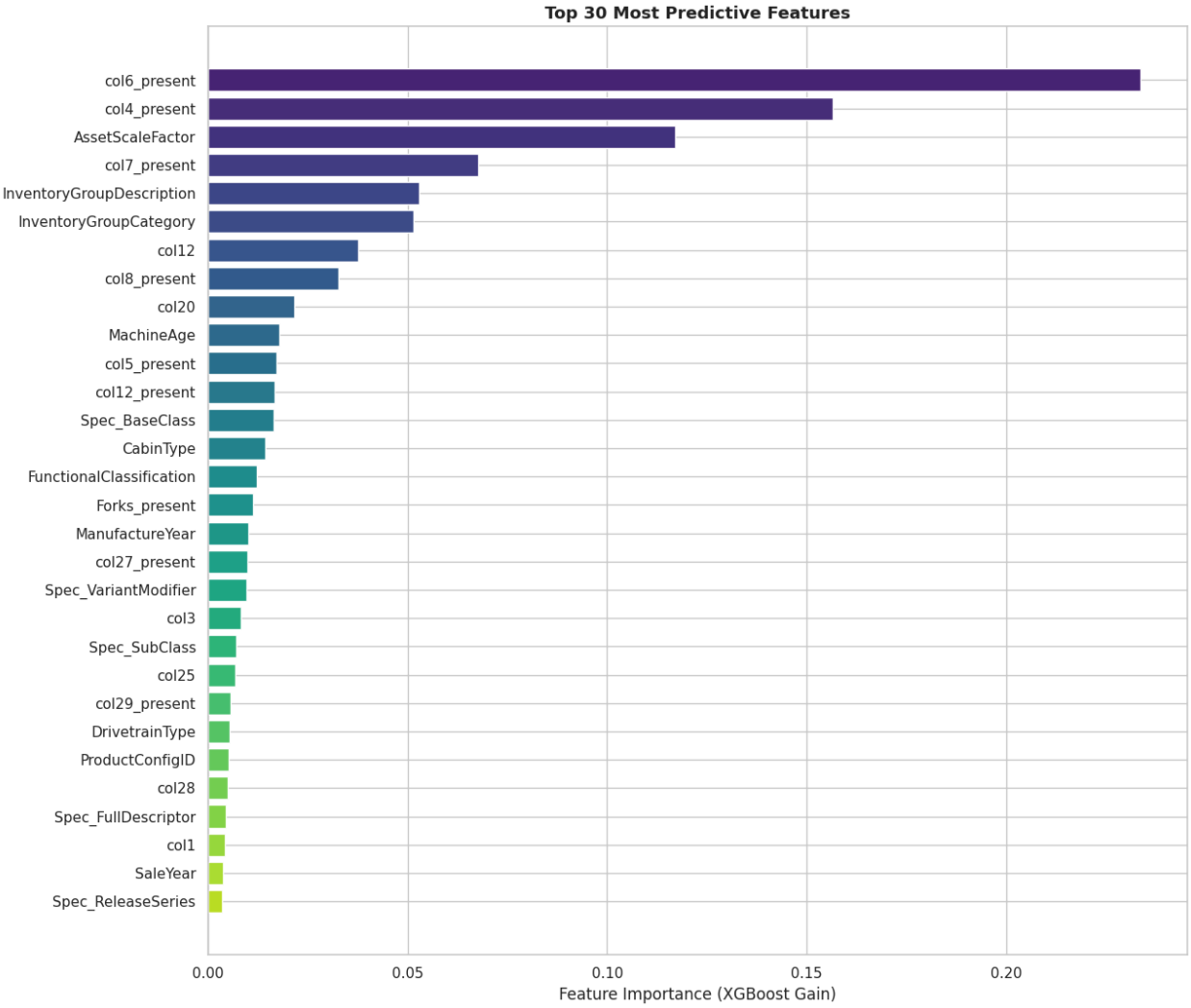

```
Error Metrics — Validation Set:
  RMSLE  : 0.2045  (competition metric)
  MAE    : $  5,399.57
  RMSE   : $  8,060.19
  R2     :      0.9052
  MedAPE :       10.73%
```

In [33]:
```
# ═══════════════════════════════════════════════════════
# 7.2  FEATURE IMPORTANCE
# ═══════════════════════════════════════════════════════
fi = pd.DataFrame({'Feature': feature_names_out,
                   'Importance': rs.best_estimator_.feature_importances_}
                  ).sort_values('Importance', ascending=False).head(30)

fig, ax = plt.subplots(figsize=(13, 11))
cmap = plt.cm.viridis(np.linspace(0.1, 0.9, len(fi)))
ax.barh(fi['Feature'][::-1], fi['Importance'][::-1], color=cmap[::-1], edgecol
ax.set_xlabel('Feature Importance (XGBoost Gain)')
ax.set_title('Top 30 Most Predictive Features', fontsize=13, fontweight='bold'
plt.tight_layout(); plt.show()

print("Top 20 Features:")
print(fi.head(20).to_string(index=False))
print("\nKey findings:")
print("  MachineAge / ManufactureYear  — depreciation is the strongest signal"
print("  OperationalHoursMeter         — more usage = lower residual value")
```

```
print("  FunctionalClassification      – machine type sets the price bracket")
print("  RegionCode                    – geographic price variation")
```



```
Top 20 Features:
                   Feature  Importance
              col6_present        0.23
              col4_present        0.16
         AssetScaleFactor         0.12
              col7_present        0.07
InventoryGroupDescription         0.05
   InventoryGroupCategory         0.05
                    col12         0.04
              col8_present        0.03
                    col20         0.02
               MachineAge         0.02
              col5_present        0.02
             col12_present        0.02
           Spec_BaseClass         0.02
                CabinType         0.01
 FunctionalClassification         0.01
            Forks_present         0.01
          ManufactureYear         0.01
             col27_present        0.01
     Spec_VariantModifier         0.01
                    col3          0.01

Key findings:
  MachineAge / ManufactureYear  — depreciation is the strongest signal
  OperationalHoursMeter         — more usage = lower residual value
  FunctionalClassification      — machine type sets the price bracket
  RegionCode                    — geographic price variation
```

8. Final Prediction & Submission

- Model: **XGBoost (tuned hyperparameters)** trained on all 138 701 samples
- Convert log1p predictions back to USD with `expm1()`
- Clip any negatives to 0 (physical constraint — no negative prices)
- Save `submission.csv` in the required format

In [35]:
```python
# ═══════════════════════════════════════════════════════════
# 8.  FINAL SUBMISSION
# ═══════════════════════════════════════════════════════════
sub = make_submission(final_model, X_test_final_proc, test_ids,
                      log_target=True, filename='submission.csv')

print("\nSubmission statistics:")
print(f"  Rows   : {len(sub):,}")
print(f"  Min    : ${sub['TargetValue'].min():>12,.2f}")
print(f"  Max    : ${sub['TargetValue'].max():>12,.2f}")
print(f"  Mean   : ${sub['TargetValue'].mean():>12,.2f}")
print(f"  Median : ${sub['TargetValue'].median():>12,.2f}")
neg = (sub['TargetValue'] < 0).sum()
print(f"  Negative predictions: {neg}  {'✅ None' if neg==0 else '⚠  clipped'}")
```

```
print(f"\n{'='*55}")
print(f"  SUBMISSION READY!")
print(f"  File : submission.csv")
print(f"  Val RMSLE : {tuned_rmsle:.4f}  (target <= 0.85)")
print(f"{'='*55}")
```

```
✅ submission.csv saved (15,000 rows)

Submission statistics:
  Rows   : 15,000
  Min    : $    7,251.37
  Max    : $  141,053.44
  Mean   : $   40,796.54
  Median : $   34,579.79
  Negative predictions: 0  ✅ None

============================================================
  SUBMISSION READY!
  File : submission.csv
  Val RMSLE : 0.2045  (target <= 0.85)
============================================================
```

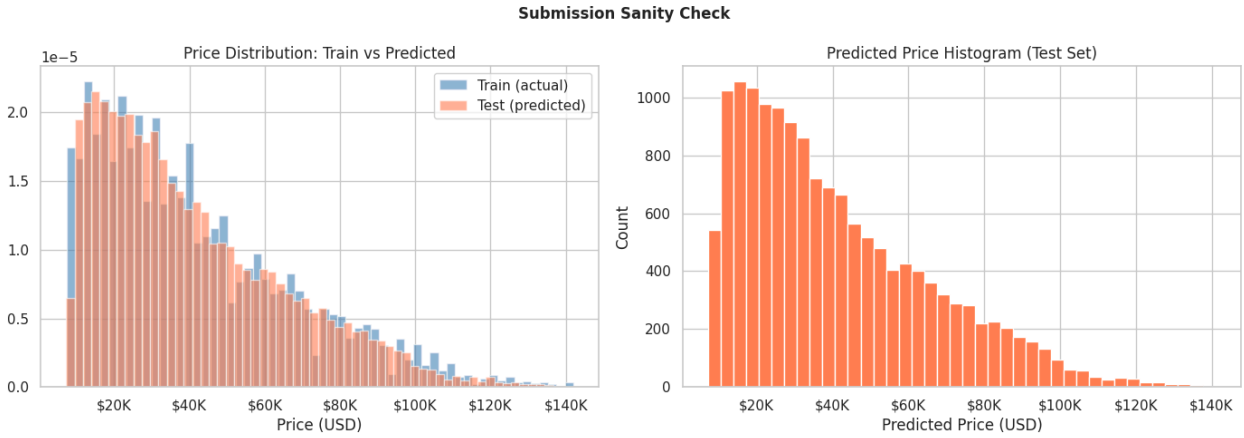
In [36]:
```
# Sanity check: test prediction distribution should mirror training
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle('Submission Sanity Check', fontsize=12, fontweight='bold')

axes[0].hist(train_df['TargetValue'], bins=60, alpha=.6, color='steelblue',
             edgecolor='white', label='Train (actual)', density=True)
axes[0].hist(sub['TargetValue'], bins=60, alpha=.6, color='coral',
             edgecolor='white', label='Test (predicted)', density=True)
axes[0].set_title('Price Distribution: Train vs Predicted')
axes[0].set_xlabel('Price (USD)')
axes[0].legend()
axes[0].xaxis.set_major_formatter(mticker.FuncFormatter(lambda x,_: f'${x/1000

axes[1].hist(sub['TargetValue'], bins=40, color='coral', edgecolor='white')
axes[1].set_title('Predicted Price Histogram (Test Set)')
axes[1].set_xlabel('Predicted Price (USD)')
axes[1].set_ylabel('Count')
axes[1].xaxis.set_major_formatter(mticker.FuncFormatter(lambda x,_: f'${x/1000

plt.tight_layout(); plt.show()

print("First 10 rows of submission.csv:")
print(sub.head(10).to_string(index=False))
```



First 10 rows of submission.csv:

TransactionID	TargetValue
1139307	65914.03
1139419	79530.84
1139482	31791.61
1139522	20816.60
1139684	12498.78
1139773	29677.71
1139779	19945.93
1139793	48282.14
1139814	67472.02
1139839	16163.36